

COMPSCI5079 (M)

Cryptography & Secure Development

Dr. Nguyen Truong

Lecturer (Assistant Professor), SoCS, University of Glasgow

Email: Nguyen.Truong@Glasgow.ac.uk

Office: SAWB 221C, Sir Alwyn Williams Building, 18 Lilybank Gardens



Lecture 1:

Fundamental Foundation for Cryptography





Lecture Outline

1. Cryptography Overview & Terminology

2. Mathematical Foundation

- Integer mod N
- Polynomial Arithmetic

3. Information Theory



Integers mod n

- Text, and indeed any input, can be converted to a series of integers.
 - Using a numbering scheme.
 - Converting to bits, creating blocks of bits, converting each block to an integer.
- Dividing by integers doesn't usually result in an integer.
- Using `integers mod  $n$` lets us define integer inverses.
 - Solve equation: $ax \equiv 1 \pmod{n}$
- If the number a and the modulus n have a common factor, then there is no inverse.
 - Choosing n as a *prime* is convenient.
- Solving equation $ax \equiv 1 \pmod{n}$ uses a trick since $n * x \equiv n \pmod{n}$ is always true.
 - Keep subtracting, reducing the number in front of x each time.
- We can take the remainder when dividing by n whenever we want, which reduces the size of numbers in our calculations.



Exclusive OR (**XOR**)



Integers mod 2

- Polynomial arithmetic, as used in cryptography, is based on `integers mod 2`.
- There are only 2 values, 0 and 1.
 - They can be stored in a single bit.
- Addition and subtraction are both the same and equal to Exclusive OR.

$0+0=0,$	$0-0=0,$	$0\oplus 0=0$
$0+1=1,$	$0-1=-1=1,$	$0\oplus 1=1$
$1+0=1,$	$1-0=1,$	$1\oplus 0=1$
$1+1=2=0,$	$1-1=0,$	$1\oplus 1=0$



XOR Operator

- The Exclusive OR (XOR) operator applies `integers mod 2` arithmetic to all bits independently.
- There are two special situations. Let a be a bit (value 0 or 1)

$$\begin{array}{ll} a \oplus a = 0 & \text{if bits the same, result is 0} \\ a \oplus 0 = a & 0 \oplus 0 = 0, 1 \oplus 0 = 1 \end{array}$$

- Two operations with the same value cancel out.
- For example, swap values in a and b with no extra variable.

$$\begin{array}{ll} a = a \oplus b & a \text{ stores } a \oplus b \\ b = a \oplus b & b \text{ stores } a \oplus b \oplus b = a \oplus 0 = a \\ a = a \oplus b & a \text{ stores } a \oplus b \oplus a = b \oplus 0 = b \end{array}$$

- In programming languages like C, C++ and Java, the Exclusive OR (XOR) operator is `^`.



Powers and Logs



Logs: some Observations

➤ The $\log$ function is the inverse of the power function.

- If $x = b^y \rightarrow y = \log_b(x)$; $x > 0$
- Special values:
 - If $x = 2^y \rightarrow y = \log_2(x)$
 - If $y = 1$ then $x = 2 \rightarrow \log_2(2) = 1$
 - If $y = 0$ then $x = 1 \rightarrow \log_2(1) = 0$

➤ Some properties: Rules for combining powers/Logs rules

- $b^x b^y = b^{x+y}$ | $\log_b(x \cdot y) = \log_b(x) + \log_b(y)$
- $1/b^x = b^{-x}$ | $\log_b(x/y) = \log_b(x) - \log_b(y)$
- $(b^x)^y = b^{xy}$ | $\log_b(x^y) = y \log_b(x)$

➤ Logs base 2 are convenient for computing.



Logs: some Observations

- $2^5 = 32$ and so $\log_2(32) = 5$
 - Similar to other powers of 2.
- 2^{10} is roughly 1,000 and so $\log_2(1000)$ is roughly 10
- 2^{20} is roughly $1,000 * 1,000 = 1,000,000$.
- 26 is between 16 and 32 and so $\log_2(26)$ is between 4 and 5.



Polynomial Arithmetic



Polynomials

- Polynomials are algebraic expressions that consist of variables and coefficients.
 - An example of a polynomial with one variable is $x^2 + x - 12$. In this example, there are three terms: x^2 , x and -12 .
- We can perform [arithmetic operations](#) such as addition, subtraction, multiplication, and also positive integer exponents for polynomial expressions but not division by variable.



Polynomials in Cryptography

- A polynomial involves terms in x , x^2 , x^3 and so on.
 - We are not interested in actually calculating x .
 - We just want to use polynomial arithmetic to produce new polynomials.
- We are interested in a special form of polynomial where the coefficients of the powers of x are integers mod 2.
 - Either 1 or 0.
- So, a typical polynomial would be
 - $1 + x^2 + x^5$
- Or, written in full.
 - $1 + 0*x + 1*x^2 + 0*x^3 + 0*x^4 + 1*x^5 + 0*x^6 \dots$



Polynomial Arithmetic

- Polynomial arithmetic is more convenient than `integers mod n` when converting to and from a bit string.
 - AES (see later) uses polynomial arithmetic.
- `Integers mod n` must be used when actual numbers are being used.
 - Public Key algorithms such as RSA and Diffie-Hellman (see later) use `integers mod n`.



Bit String & Polynomial

- Converting from a bit string to one of these polynomials is very easy.
- The bits, starting with the least significant bit, are the coefficients of the powers of x .

$$1101_2 \rightarrow x^3 + x^2 + 1$$

- Converting back is also straightforward.

$$x^3 + x^2 + 1 \rightarrow 1101_2$$

- The operations, $+$, $-$, $*$, $\%$, for encryption and decryption operations can be done with polynomial arithmetic.



Polynomial Arithmetic

- Addition and subtraction are straightforward.
- Subtraction is the same as addition! Both are XOR

$$\begin{array}{rcccc}
 & 1 & + & & x^2 & + & x^3 \\
 + & & & x & + & & x^3 \\
 = & 1 & + & x & + & x^2 &
 \end{array}$$

- Multiplication is also straightforward but leads to bigger polynomials.

$$\begin{array}{rccccccc}
 & & (1+x^2+x^3) & (x+x^3) & = & & \\
 & x & + & & x^3 & + & x^4 \\
 + & & & x^3 & + & x^5 & + & x^6 \\
 = & x & + & & x^4 & + & x^5 & + & x^6
 \end{array}$$



Remainder after Division

- We can reduce the size of polynomials produced by multiplication by taking the remainder after dividing one polynomial by another.
 - The same trick as `integers mod n`.
- If we wanted the highest power to be x^3 then we divide by a polynomial with the highest power x^4
- Polynomial division is complicated, even though taught in high school maths
- It is a lot easier with polynomials based on `integers mod 2`!



Prime Polynomials

- Some polynomials are 'prime'
 - They can't be written as factors of smaller polynomials.
 - They are called **irreducible polynomials**.
- If we do polynomial arithmetic mod (an irreducible polynomial), then all polynomials will have inverses.
- Some **irreducible polynomials**:

3 bit: $x^3 + x + 1$

4 bit: $x^4 + x + 1$

6 bit: $x^6 + x + 1$

8 bit: $x^8 + x^4 + x^3 + x + 1$



3-bit Polynomials

- The irreducible polynomial is $IP = x^3 + x + 1$.
- The eight possible polynomials are:

$$0 = 000_2 = 0$$

$$1 = 001_2 = 1$$

$$2 = 010_2 = x$$

$$3 = 011_2 = x + 1$$

$$4 = 100_2 = x^2$$

$$5 = 101_2 = x^2 + 1$$

$$6 = 110_2 = x^2 + x$$

$$7 = 111_2 = x^2 + x + 1$$



Polynomial Inverses

- The algorithm for calculating polynomial inverses is similar to that for integers mod n .
 - Let p be a polynomial.
 - Let z be the inverse of p , also a polynomial.
 - $pz = 1 \text{ mod irreducible polynomial}$.
 - Polynomial arithmetic is used.
- The calculations are more complex, and so for small polynomials, it is easier to create a [multiplication table](#).
 - I have done the polynomial arithmetic for 3-bit polynomials, leading to the following table.
- The inverses are found by looking for [1](#) in each row.



Multiplication table for 3-bit Polynomials

	1	2	3	4	5	6	7	Inverse	
1:	1	2	3	4	5	6	7	1	$(1 * 1 = 1)$
2:	2	4	6	3	1	7	5	5	$(2 * 5 = 1)$
3:	3	6	5	7	4	1	2	6	$(3 * 6 = 1)$
4:	4	3	7	6	2	5	1	7	$(4 * 7 = 1)$
5:	5	1	4	2	7	3	6	2	$(5 * 2 = 1)$
6:	6	7	1	5	3	2	4	3	$(6 * 3 = 1)$
7:	7	5	2	1	6	4	3	4	$(7 * 4 = 1)$



Example: $5 * 6 \bmod 3\text{-bit IP}$

$$5 = 101_2 = x^2 + 1$$

$$6 = 110_2 = x^2 + x$$

$$P = (x^2 + 1)(x^2 + x) = x^4 + x^3 + x^2 + x$$

$$\text{Irreducible polynomial IP} = x^3 + x + 1$$

We need to calculate: $P \bmod \text{IP}$

1. Remove the power of x^4 : $Q = P - x * \text{IP}$

$$\begin{array}{r} x^4 + x^3 + x^2 + x \\ - x^4 + + x^2 + x \\ \hline x^3 \end{array}$$

2. Remove the power of x^3 : $R = Q - \text{IP}$

$$\begin{array}{r} x^3 \\ - x^3 + x + 1 \\ \hline x + 1 \\ = 011_2 = 3 \end{array}$$



8-bit Polynomials

- Many algorithms work with 8-bit polynomials since they represent a single byte.
- Results can be stored in a lookup table.
 - There are $2^8 = 256$ different polynomials.
 - A table is needed for multiplication:
 - There are $2^{16} = 64k$ values.
 - Also, for inverses, there are $2^8 = 256$ values.
 - Each value is 1 byte.
- This is easy to store in hardware, typically on a CPU chip.



Galois Fields

- The mathematical name for using integers mod n and polynomials mod an irreducible polynomial is computing with *Galois Fields* (GF) .
 - Doesn't it seem a lot harder already?
- Integers mod n are $GF(n)$.
- Polynomials of degree n (i.e. $n=8$ for bytes) are $GF(2^n)$.
- Don't be put off by terminology!



Quizzes

1. How is polynomial arithmetic similar and how does it differ from using integers mod n ? In what way is it better? When must integers mod n be used rather than polynomial arithmetic?
2. Convert 1010_2 and 0110_2 to polynomials, add them up, and convert the answer back to bit strings.
3. Now multiply them, using the irreducible polynomial $x^4 + x + 1$



Lecture Summary



Polynomial Arithmetic

- Exclusive OR is a useful operation in cryptography
 - Two applications of the same value cancel out.
- Polynomial arithmetic usually uses single bits (0 or 1) as coefficients of powers of x .
- This makes it easy to convert from bits to polynomials.
 - Each bit corresponds to a coefficient in the polynomial.
- It is common to split the bit string input into individual 8-bit bytes.
 - This involves arithmetic with polynomials with the highest power x^7 .
- Adding and subtracting polynomials is easy.
 - Each coefficient is combined using Exclusive OR.
 - If both coefficients are the same, the answer is 0.
 - If they are different the answer is 1.



Polynomial Arithmetic

- Multiplying polynomials produces longer polynomials.
- We can divide by a 'prime' (irreducible) polynomial and keep the remainder.
 - Like integers mod n .
- It is convenient to create a multiplication table so that multiplication calculations just need a fast table lookup.